Guia de uso

Respostas comentadas

Atividades de sala até a P2

18 questões · respostas de prova · explicações e exemplos

Este caderno cobre as 18 questões da seção “Questões de sala até a P2” de AtividadesSala.txt. Cada questão apresenta primeiro uma resposta em formato de prova e depois uma explicação. Os enunciados abaixo foram normalizados para corrigir a notação, sem ocultar as ambiguidades do arquivo.

Convenções: n = |V| e m = |E|. Caminhos mínimos minimizam a soma dos pesos, salvo quando o enunciado pede quantidade de arestas. Em grafos direcionados, respeitamos as setas. Uma AGM exige grafo não direcionado e conexo; em grafos desconexos, usamos floresta geradora mínima. Para grafos de linha, consideramos grafos simples não direcionados. “Hamiltoniano” significa possuir um ciclo hamiltoniano, com pelo menos três vértices.

Pontos de atenção: a questão 6 não contém seu grafo no arquivo; por isso, há um exemplo ilustrativo, e não uma resolução numérica do grafo original. A questão 13 está ambígua; suas interpretações são tratadas separadamente. A questão 11(iii) exige respeitar tamanhos dos grupos: simplesmente cortar as maiores arestas não garante isso.

> **Leia primeiro a resposta de prova; use a explicação para entender as escolhas.**

Os desenhos são esquemáticos e têm legendas. Use os marcadores do PDF ou o índice para localizar cada questão. O material cobre somente a seção P2, conforme solicitado.

Índice

Questão 01 — Menor quantidade de arestas 3

Questão 02 — Caminho mínimo com pesos inteiros positivos 4

Questão 03 — Caminho mínimo com pesos reais positivos 5

Questão 04 — Caminho mínimo com pesos reais quaisquer 6

Questão 05 — Menor caminho crescente 7

Questão 06 — Dijkstra passo a passo 8

Questão 07 — Rota com menor chance de perda 9

Questão 08 — K grupos por afinidade 10

Questão 09 — Transformar Dijkstra em Prim 11

Questão 10 — Dois algoritmos de AGM 12

Questão 11 — AGM, Dijkstra e tamanho dos grupos 13

Questão 12 — Zerar uma aresta sem refazer caminhos mínimos 15

Questão 13 — Remoção de aresta e floresta mínima 16

Questão 14 — A aresta de menor peso está na AGM? 17

Questão 15 — Construir um ciclo euleriano 18

Questão 16 — Euleriano, hamiltoniano e grafo de linha 19

Questão 17 — Quando o grafo de linha é euleriano ou hamiltoniano? 21

Questão 18 — Aplicações de grafos eulerianos 23

Questão 01 — Menor quantidade de arestas

ENUNCIADO

Em um grafo direcionado, encontre um caminho de u para v com a menor quantidade de arestas.

RESPOSTA DE PROVA

Execute uma busca em largura (BFS) a partir de u, considerando apenas as arestas de saída de cada vértice. Inicialize d(u) = 0 e d(x) = ∞ para os demais vértices. Ao descobrir pela primeira vez um vértice y por uma aresta (x,y), defina d(y) = d(x) + 1, registre pai(y) = x e insira y na fila. Se v for alcançado, reconstrua o caminho seguindo os predecessores de v até u; caso contrário, informe que não existe caminho.

A BFS processa os vértices por camadas de distância: primeiro os alcançáveis com zero arestas, depois com uma, com duas e assim por diante. Assim, a primeira descoberta de v ocorre por um caminho com a menor quantidade de arestas. O tempo é O(n + m) e o espaço auxiliar é O(n), usando listas de adjacência.

EXPLICAÇÃO

Aqui não importa se uma aresta é “cara”: cada aresta vale um passo. A fila da BFS impede que um caminho longo seja escolhido antes de um mais curto em número de arestas.

Exemplo: se existem A → B → D e A → C → E → D, a BFS encontra D com distância 2, pelo caminho A → B → D. Se u = v, a resposta é o caminho vazio, com zero arestas.

Questão 02 — Caminho mínimo com pesos inteiros positivos

ENUNCIADO

Em um grafo direcionado com w: E → Z+, encontre um caminho de u para v com a menor soma dos pesos.

RESPOSTA DE PROVA

Execute Dijkstra com origem u. Inicialize d(u) = 0, d(x) = ∞ para x ≠ u e os predecessores como indefinidos. Mantenha os vértices em uma fila de prioridade pela distância conhecida. Ao retirar um vértice x de menor distância ainda não finalizado, relaxe suas arestas de saída (x,y):

> **d(y) ← min{d(y), d(x) + w(x,y)}.**

Sempre que houver melhora, atualize o predecessor de y e sua prioridade. A distância de v é definitiva quando ele é finalizado. Se continuar infinita, não existe caminho; caso contrário, reconstrua o caminho pelos predecessores.

Como os pesos são positivos, qualquer prolongamento de um caminho não diminui seu custo. Isso garante a propriedade de Dijkstra: ao finalizar o vértice de menor distância provisória, sua distância já é mínima. Com heap binário e listas de adjacência, o tempo é O((n + m) log n) e o espaço auxiliar é O(n), usando uma fila com atualização de prioridades.

EXPLICAÇÃO

O algoritmo compara custos totais desde a origem, não apenas o peso da próxima aresta. Uma rota com duas arestas pode ser melhor que uma rota direta: A → B de peso 2 e B → C de peso 3 totalizam 5; se A → C pesa 9, a rota por B é melhor.

O vetor d guarda uma distância para cada destino. Ele não deve ser somado para obter o custo de um caminho individual.

Questão 03 — Caminho mínimo com pesos reais positivos

ENUNCIADO

Em um grafo direcionado com w: E → R+, encontre um caminho de u para v com a menor soma dos pesos.

RESPOSTA DE PROVA

Utilize Dijkstra, exatamente como na questão 2. Inicialize d(u) = 0 e as demais distâncias com infinito. Repetidamente, finalize o vértice de menor distância provisória e relaxe suas arestas de saída. Atualize os predecessores quando houver melhora e reconstrua o caminho até v ao final.

> **Se d(x) + w(x,y) < d(y), atualize d(y) e pai(y).**

A correção não depende de os pesos serem inteiros, mas de serem não negativos. Portanto, pesos reais positivos também satisfazem a hipótese do algoritmo. O tempo com heap binário é O((n + m) log n).

EXPLICAÇÃO

Pesos como 1,5 ou 2,75 não mudam a lógica: basta somá-los e comparar os custos. Por exemplo, A → B com 1,5 e B → C com 2,0 custam 3,5; isso é melhor que uma aresta direta A → C com peso 4,0.

Não é necessário arredondar os pesos para inteiros. Em uma implementação numérica, apenas se deve considerar a precisão da representação utilizada.

Questão 04 — Caminho mínimo com pesos reais quaisquer

ENUNCIADO

Em um grafo direcionado com w: E → R, encontre o menor caminho entre u e v.

RESPOSTA DE PROVA

Como podem existir pesos negativos, utilize Bellman–Ford. Inicialize d(u) = 0 e as demais distâncias com infinito. Faça até n − 1 passadas sobre todas as arestas, relaxando (x,y) quando d(x) for finita e d(x) + w(x,y) < d(y). Guarde os predecessores. Se uma passada não produzir alterações, pode-se interromper a repetição.

Depois, examine novamente as arestas. Para cada aresta ainda relaxável com origem alcançável a partir de u, marque seu destino e todos os vértices alcançáveis a partir dele. Se v estiver marcado, existe um ciclo negativo alcançável de u a partir do qual se pode chegar a v. Nesse caso, não há menor custo finito para os percursos de u a v.

Se v não for afetado e d(v) for finita, reconstrua um caminho mínimo. Se d(v) = ∞, não existe caminho. Na ausência de ciclos negativos relevantes, um caminho mínimo pode ser escolhido com no máximo n − 1 arestas; as passadas de relaxamento consideram essas possibilidades. O tempo é O(nm), mais O(n + m) para propagar as marcações.

EXPLICAÇÃO

Dijkstra não é seguro com pesos negativos. Bellman–Ford permite que distâncias sejam corrigidas em várias passadas.

Um ciclo de peso −2 pode ser repetido para reduzir o custo em 2 a cada volta. Mas ele só impede uma resposta finita de u até v se for alcançável a partir de u e permitir chegar a v. Um ciclo negativo em uma região sem acesso a v não invalida esse destino.

A expressão “não há menor custo” considera percursos que podem repetir vértices. Se a questão exigir caminhos simples mesmo com ciclos negativos, essa restrição muda o problema e Bellman–Ford não o resolve em geral.

Questão 05 — Menor caminho crescente

ENUNCIADO

Em um grafo direcionado com pesos inteiros positivos, encontre um caminho de u para v de menor custo cujos pesos sejam crescentes.

RESPOSTA DE PROVA

Considere “crescente” como estritamente crescente. Ordene as arestas por peso e processe juntas aquelas que possuem o mesmo peso. Mantenha melhor[x], o menor custo de um caminho crescente de u até x que utiliza apenas os pesos já processados. Inicialize melhor[u] = 0 e melhor[x] = ∞ para x ≠ u.

Para cada grupo de peso q, calcule, para toda aresta (a,b) desse grupo com melhor[a] finito, o candidato melhor[a] + q. Guarde os candidatos sem atualizar melhor durante o exame do grupo. Somente depois de examinar todas as arestas de peso q, aplique as melhorias.

> **candidato(a,b) = melhor[a] + q, com último peso anterior < q.**

Ao final, melhor[v] é o menor custo crescente, ou infinito se não houver caminho. Para reconstruir a resposta, armazene registros de caminho com referências fixas aos registros anteriores, preservando a versão usada por cada candidato.

Por indução nos grupos de pesos, antes de processar q, melhor[a] representa o melhor caminho cujo último peso é menor que q. Portanto, acrescentar uma aresta de peso q preserva o crescimento. Todo caminho crescente é considerado na etapa de sua última aresta. O tempo é O(m log m + n), dominado pela ordenação, e o espaço auxiliar é O(n + m).

EXPLICAÇÃO

Não basta usar o Dijkstra comum com uma única distância por vértice: precisamos controlar o peso da última aresta. A ordenação oferece uma forma direta de fazer isso.

Para pesos 2, 5 e 9, a sequência 2 → 5 → 9 é permitida. A sequência 2 → 2 → 5 não é estritamente crescente. Por isso, arestas com pesos iguais precisam ser avaliadas em lote, sem usar resultados produzidos dentro do mesmo lote.

Também é possível aplicar Dijkstra aos estados (vértice, último peso). A condição para atravessar uma aresta de peso q é q > último peso. Se “crescente” permitir igualdade, a regra deve ser ajustada; o processamento em lotes descrito acima é para crescimento estrito.

Questão 06 — Dijkstra passo a passo

ENUNCIADO

Encontre o menor caminho de A para todos os vértices, mostrando as etapas.

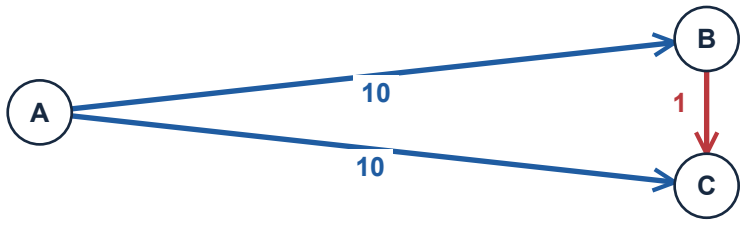
RESPOSTA DE PROVA

O arquivo não apresenta o grafo desta questão. Sem suas arestas e seus pesos, não é possível calcular as distâncias do exercício original. O procedimento é executar Dijkstra a partir de A, registrando a cada etapa o vértice finalizado, as distâncias provisórias e os predecessores.

A seguir, usamos somente um exemplo ilustrativo: A → B com peso 10; A → C com peso 10; B → C com peso 1. Inicialmente, d(A) = 0 e as outras distâncias são infinitas. Ao processar A, obtemos d(B) = 10 e d(C) = 10. Escolhendo B primeiro no empate, o candidato para C é 10 + 1 = 11; como 11 > 10, nenhuma alteração é feita. Ao processar C, o algoritmo termina.

Etapa	Vértice processado	d(A)	d(B)	d(C)
Inicial	nenhum	0	∞	∞
1	A	0	10	10
2	B	0	10	10
3	C	0	10	10

Exemplo ilustrativo da questão 6



EXPLICAÇÃO

Os menores caminhos do exemplo são A → B, de custo 10, e A → C, de custo 10. A aresta B → C não melhora o caminho até C, pois a rota A → B → C custa 11.

A árvore de caminhos mínimos possui A → B e A → C; a soma de suas arestas é 20. Isso não significa que o menor caminho até C custa 20: ele custa 10. Dijkstra calcula distâncias individuais, e não uma soma única de todas elas.

Questão 07 — Rota com menor chance de perda

ENUNCIADO

Cada aresta possui uma probabilidade de perda constante e independente das demais. Escolha uma rota de A para B que minimize a chance de perda da mensagem.

RESPOSTA DE PROVA

Seja p(e) a probabilidade de perda em e e q(e) = 1 − p(e) sua probabilidade de sucesso. Pela independência, a probabilidade de a mensagem percorrer uma rota P com sucesso é:

> **$S(P) = \prod q(e)$, para e em P.**

Minimizar a probabilidade de perda 1 − S(P) equivale a maximizar S(P). Para q(e) > 0, atribua à aresta o custo c(e) = −ln(q(e)). Como 0 < q(e) ≤ 1, temos c(e) ≥ 0. Então:

> **$-\ln(S(P)) = \sum c(e)$, para e em P.**

Assim, execute Dijkstra de A para B com os custos c. A rota com menor soma desses custos possui a maior probabilidade de sucesso. Sua probabilidade de perda é 1 − exp(−d(B)). Arestas com p(e) = 1 possuem custo infinito e podem ser ignoradas na busca por uma rota com sucesso positivo. O tempo é O((n + m) log n).

Se não houver rota formada somente por arestas com p(e) < 1, verifique a alcançabilidade no grafo original: se B for alcançável, toda rota tem perda 1; caso contrário, não existe rota.

EXPLICAÇÃO

Não devemos somar probabilidades de perda. Por exemplo, duas arestas com perda 10% têm sucesso total 0,9 × 0,9 = 0,81 e perda 19%, e não 20%.

Se uma rota direta tem perda 25%, seu sucesso é 75%. A rota de duas arestas com perda de 10% em cada uma é melhor, pois seu sucesso é 81%. O logaritmo converte a multiplicação em soma para permitir o uso do Dijkstra.

Questão 08 — K grupos por afinidade

ENUNCIADO

Modele n alunos e suas afinidades por um grafo e obtenha K grupos a partir de uma árvore geradora, removendo K − 1 arestas.

RESPOSTA DE PROVA

Represente cada aluno por um vértice e cada afinidade a(i,j) por uma aresta, em que valores maiores significam maior afinidade. Construa uma árvore geradora máxima, usando Kruskal com as arestas em ordem decrescente. Em seguida, remova as K − 1 arestas de menor afinidade da árvore. As K componentes resultantes são os grupos, para 1 ≤ K ≤ n.

Equivalentemente, converta afinidade em distância por d(i,j) = M − a(i,j), com M maior ou igual à maior afinidade. Calcule uma AGM e remova suas K − 1 maiores arestas. Cada remoção de uma aresta de uma árvore aumenta em um o número de componentes, produzindo exatamente K grupos. O custo é O(m log m), usando Kruskal.

Afinidade: para K = 2, corte a ligação mais fraca



EXPLICAÇÃO

Se o peso mede proximidade, queremos manter pesos altos e cortar os baixos. Se mede distância, queremos manter pesos baixos e cortar os altos.

Exemplo: uma árvore contém AB com afinidade 9, BC com afinidade 2 e CD com afinidade 8. Para formar dois grupos, corte BC. Os grupos ficam {A,B} e {C,D}.

Esse método é um agrupamento por ligação simples. Com pesos de distância, ele maximiza a menor distância entre grupos. Não garante maximizar a soma de todas as afinidades internas nem que todos os pares de um grupo tenham afinidade alta. Para exigir outra função objetivo, é necessário defini-la.

Questão 09 — Transformar Dijkstra em Prim

ENUNCIADO

Altere a estrutura do algoritmo de Dijkstra para encontrar uma AGM de um grafo conexo não direcionado com pesos positivos.

RESPOSTA DE PROVA

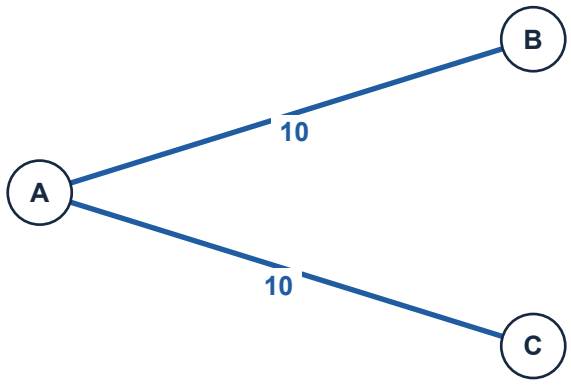
Substitua a distância acumulada usada por Dijkstra por uma chave que representa o peso da aresta mais leve que conecta cada vértice à árvore já construída. Inicialize chave[r] = 0 para uma raiz r e chave[v] = ∞ para os demais vértices.

A cada etapa, retire da fila o vértice x de menor chave ainda fora da árvore. Se x não for a raiz, inclua a aresta entre x e seu predecessor. Para cada vizinho y ainda fora da árvore, se w(x,y) < chave[y], atualize chave[y] = w(x,y) e pai[y] = x.

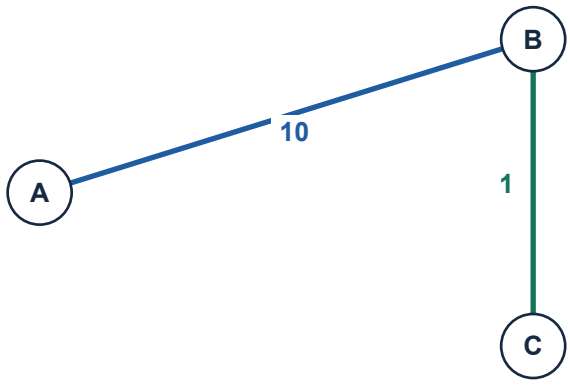
> **Dijkstra compara d(x) + w(x,y); Prim compara apenas w(x,y).**

O algoritmo obtido é Prim. Em cada etapa, escolhe uma aresta de menor peso que atravessa o corte entre os vértices já incluídos e os restantes. Pela propriedade do corte, essa escolha é segura para construir uma AGM. O tempo com heap binário é O(m log n), para grafo conexo.

Dijkstra desde A: total 20



Uma AGM: total 11



EXPLICAÇÃO

Dijkstra responde “qual o menor custo desde a origem até y?”. Prim responde “qual a ligação mais barata de y com a árvore atual?”.

No triângulo AB = 10, AC = 10 e BC = 1, Dijkstra partindo de A pode produzir AB e AC, total 20. Prim escolhe uma aresta de peso 10 e depois BC, de peso 1, total 11. Uma árvore de caminhos mínimos não precisa ser uma AGM.

Questão 10 — Dois algoritmos de AGM

ENUNCIADO

Apresente dois algoritmos para encontrar uma AGM de um grafo conexo não direcionado com pesos positivos.

RESPOSTA DE PROVA

Primeiro algoritmo: Prim. Escolha uma raiz e mantenha um conjunto S de vértices já incluídos. A cada etapa, adicione uma aresta de menor peso com um extremo em S e outro fora de S, incorporando o novo vértice. Termine quando todos os vértices tiverem sido incluídos. A escolha é segura pela propriedade do corte. O tempo com heap binário é O(m log n).

Segundo algoritmo: Kruskal. Ordene todas as arestas em ordem crescente e inicialize cada vértice em uma componente própria. Examine as arestas nessa ordem e inclua uma aresta somente se seus extremos estiverem em componentes diferentes; depois, una essas componentes. Use uma estrutura de conjuntos disjuntos para essas operações. Pare ao selecionar n − 1 arestas.

Kruskal nunca cria ciclos e sempre escolhe uma aresta de menor peso entre componentes diferentes, uma escolha segura pela propriedade do corte. Como o grafo é conexo, obtém uma árvore geradora mínima. Seu tempo é O(m log m), dominado pela ordenação.

EXPLICAÇÃO

Prim cresce uma única árvore. Kruskal começa com vários componentes isolados e vai juntando-os.

Exemplo: AB = 1, BC = 2 e AC = 5. Prim a partir de A escolhe AB e depois BC. Kruskal escolhe AB e BC nessa mesma ordem e rejeitaria AC por criar ciclo. A AGM tem custo 3.

Questão 11 — AGM, Dijkstra e tamanho dos grupos

ENUNCIADO

(i) Apresente algoritmos de AGM para pesos reais e um exemplo. (ii) Compare a árvore de Dijkstra com a AGM. (iii) Ao cortar uma AGM em K grupos, garanta que cada grupo tenha pelo menos s vértices.

RESPOSTA DE PROVA

(i) Utilize Prim ou Kruskal. Ambos funcionam com pesos reais, inclusive negativos, pois suas escolhas usam a comparação entre arestas e a propriedade do corte. No triângulo AB = −2, BC = 1 e AC = 4, ambos encontram {AB, BC}, com custo −1. Se o grafo não for conexo, o resultado apropriado é uma floresta geradora mínima.

(ii) Não se pode afirmar que a árvore de Dijkstra seja sempre diferente da AGM, nem que seja sempre igual. Com pesos não negativos, no triângulo AB = 10, AC = 10 e BC = 1, a árvore de Dijkstra desde A tem custo 20, enquanto a AGM tem custo 11. Em contrapartida, se o grafo já é uma árvore de pesos positivos, ambas coincidem com o próprio grafo. Se houver pesos negativos, Dijkstra não é aplicável em geral.

(iii) Não basta remover as K − 1 maiores arestas. Uma solução exata é enumerar os subconjuntos R de K − 1 arestas de T. Para cada R, calcule as componentes de T − R e seus tamanhos. Aceite somente os cortes que produzem K componentes, todas com pelo menos s vértices. Retorne uma partição válida ou informe que ela não existe. Se também houver uma função objetivo, compare os cortes válidos segundo essa função.

> **Condição necessária: n ≥ K · s. Ela não é suficiente.**

Como T é árvore, toda partição em K componentes obtida por remoção de arestas corresponde a exatamente K − 1 cortes. Assim, a enumeração testa todas as possibilidades e é correta. Seu tempo é O(n · C(n − 1, K − 1)); é uma solução exata, mas pode ser cara. Para entradas maiores, pode-se usar programação dinâmica sobre a árvore com estados de tamanhos e quantidade de grupos.

Questão 11 — AGM, Dijkstra e tamanho dos grupos · continuação

EXPLICAÇÃO

AGM minimiza a soma das arestas selecionadas. Dijkstra minimiza separadamente as distâncias desde uma origem. São objetivos diferentes.

Na parte (iii), exigir pelo menos s vértices pode impedir os cortes das maiores arestas. Considere uma estrela com um centro e cinco folhas: há seis vértices. Para K = 2 e s = 2, temos 6 ≥ 4, mas qualquer corte de uma aresta isola uma folha. Portanto, não existe partição válida da árvore em dois grupos conexos com pelo menos dois vértices cada.

Para a prova, a enumeração deixa explícita a garantia: um corte só é aceito depois que todos os tamanhos foram conferidos. A condição vale para grupos que são componentes da árvore; permitir grupos desconexos muda o problema.

Questão 12 — Zerar uma aresta sem refazer caminhos mínimos

ENUNCIADO

Em um grafo direcionado com pesos positivos, zere o peso de uma aresta e encontre o menor caminho de u para v sem executar novamente um algoritmo de caminho mínimo após essa mudança.

RESPOSTA DE PROVA

Antes da alteração, execute Dijkstra a partir de u, obtendo D_u[x] = dist(u,x). Execute também Dijkstra a partir de v no grafo transposto, obtendo D_v[x] = dist(x,v) no grafo original. Guarde predecessores para reconstruir os caminhos.

Se a aresta escolhida for e = (a,b), seu novo peso será zero. A nova distância é:

> **D_nova(u,v) = min{D_u[v], D_u[a] + D_v[b]}.**

O primeiro termo corresponde a não depender da redução de peso. O segundo corresponde a chegar até a, atravessar (a,b) gratuitamente e seguir de b até v. Desconsidere termos com distâncias infinitas.

Para justificar, um menor percurso após a mudança pode ser escolhido sem ciclos, pois os pesos são não negativos. Se não usa e, seu custo é pelo menos D_u[v]. Se usa e, seu prefixo e seu sufixo custam pelo menos D_u[a] e D_v[b]. Por outro lado, os caminhos pré-calculados fornecem percursos com custo no máximo cada candidato após a redução. Portanto, a fórmula é exata.

Se for permitido escolher qual aresta zerar, minimize D_u[a] + D_v[b] sobre todas as arestas (a,b), comparando também com D_u[v]. O pré-processamento custa O((n + m) log n); uma consulta sobre uma aresta fixa custa O(1), e escolher a melhor entre todas custa O(m).

EXPLICAÇÃO

Você prepara duas tabelas antes de mudar o grafo: o custo de sair de u até qualquer vértice e o custo de qualquer vértice até v. Depois, só consulta essas tabelas.

Exemplo: A → B custa 4, B → C custa 7 e C → D custa 2. Se zerarmos B → C, o novo custo é 4 + 0 + 2 = 6. Sem essa preparação, apenas conhecer uma árvore de caminhos mínimos pode não fornecer toda a informação necessária para responder a qualquer alteração.

Ao reconstruir pelo prefixo, pela aresta zerada e pelo sufixo, podem surgir repetições; ciclos de custo não negativo podem ser removidos sem prejudicar o custo mínimo.

Questão 13 — Remoção de aresta e floresta mínima

ENUNCIADO

O arquivo mistura a notação da AGM T, de uma aresta e e do grafo G − {e}, sem deixar claro qual objeto deve ser preservado. A seguir estão as interpretações relevantes, sem alterar internamente Prim ou Kruskal.

RESPOSTA DE PROVA

Interpretação 1: encontrar uma floresta geradora mínima de G − {e}. Remova e do grafo e execute Kruskal normalmente. Ele produz uma árvore mínima em cada componente. Alternativamente, execute Prim em cada componente. Se o grafo permanecer conexo, o resultado será uma AGM; caso contrário, uma floresta. Não é necessário atribuir peso negativo a e, pois ela não estará no grafo de saída.

Se for permitido aproveitar a AGM T original: quando e não pertence a T, mantenha T, que continua mínima. Quando e pertence a T, sua retirada separa T em duas partes. Se existir outra aresta de G ligando essas partes, acrescente a de menor peso; isso produz uma AGM de G − {e}. Se não existir, e era uma ponte de G e T − {e} já é uma floresta geradora mínima.

Interpretação 2, discutida em sala: exigir uma árvore T que contenha e e, simultaneamente, que F = T − {e} seja uma floresta geradora mínima de G − {e}. Supondo G conexo, isso é possível se, e somente se, e for uma ponte do grafo original.

Para usar Prim sem alterá-lo, atribua temporariamente a e um peso menor que o de qualquer outra aresta e execute Prim. Por ser a única aresta de menor peso, e pertence à AGM obtida. Restaure seu peso original: a árvore é mínima entre as que contêm e, pois todas sofreram a mesma alteração de custo.

Se e for ponte, toda árvore geradora já precisa contê-la e sua retirada deixa árvores mínimas nas duas componentes. Se não for ponte, G − {e} permanece conexo, mas T − {e} possui duas componentes e n − 2 arestas; portanto, não é uma floresta geradora mínima desse grafo conexo. Nesse caso, a segunda interpretação não tem solução.

EXPLICAÇÃO

“e é ponte” deve ser verificado no grafo original. Em uma árvore, toda aresta é ponte; verificar apenas na árvore não distingue os casos.

Também não se deve confundir “encontrar uma árvore mínima obrigada a conter e” com “a retirada de e produzir uma floresta mínima do grafo sem e”. A primeira tarefa é resolvida pelo peso temporariamente muito baixo. A segunda inclui uma condição adicional, que exige e ser ponte.

Na implementação, use um número finito menor que todos os pesos, em vez de −∞. O termo −∞ é apenas uma maneira informal de representar a prioridade absoluta dessa aresta.

Questão 14 — A aresta de menor peso está na AGM?

ENUNCIADO

Uma aresta e de peso mínimo do grafo necessariamente pertence à AGM?

RESPOSTA DE PROVA

Considere um grafo conexo, não direcionado e sem laços. Uma aresta e de peso globalmente mínimo pertence a pelo menos uma AGM. Se for a única aresta com peso mínimo, pertence a toda AGM.

De fato, tome uma AGM T que não contenha e. Ao adicionar e, surge um ciclo. Escolha uma aresta f desse ciclo diferente de e. Como e tem peso mínimo, $w(e) \leq w(f)$. A troca $T' = T + e - f$ produz uma árvore geradora com $w(T') \leq w(T)$. Pela minimalidade de T, T' também é uma AGM, agora contendo e.

Se e for a única aresta de peso mínimo, então $w(e) < w(f)$, e a troca produziria uma árvore estritamente mais barata. Isso contradiz a existência de uma AGM que não contenha e. Portanto, nesse caso e está em todas as AGMs.

EXPLICAÇÃO

Com empates, a expressão “está na AGM” precisa ser esclarecida. No triângulo em que todas as arestas pesam 1, qualquer par de arestas forma uma AGM. Uma aresta específica aparece em algumas, mas não em todas.

Se uma aresta pesa 1 e todas as demais pesam mais que 1, ela é obrigatória em qualquer AGM. Não é necessário que todos os pesos do grafo sejam distintos; basta que essa aresta seja a única de menor peso.

Questão 15 — Construir um ciclo euleriano

ENUNCIADO

Dado um grafo não direcionado euleriano, produza um ciclo euleriano.

RESPOSTA DE PROVA

Utilize o algoritmo de Hierholzer. Escolha um vértice s de grau positivo e coloque-o em uma pilha. Enquanto a pilha não estiver vazia, observe seu topo v. Se v possuir uma aresta ainda não usada, marque essa aresta como usada e empilhe seu outro extremo. Se v não possuir arestas disponíveis, desempilhe v e acrescente-o à lista de saída. Inverta a lista ao final.

Cada aresta deve possuir um identificador único para que suas duas ocorrências nas listas de adjacência sejam reconhecidas como a mesma aresta. Assim, nenhuma aresta é usada duas vezes.

Como todos os graus são pares e os vértices de grau positivo pertencem à mesma componente, os percursos fechados formados podem ser combinados em um único ciclo que contém todas as arestas. O resultado possui m + 1 ocorrências de vértices, com o primeiro igual ao último. Com listas de adjacência e controle das posições examinadas, o tempo é O(n + m).

EXPLICAÇÃO

O objetivo é passar por cada aresta exatamente uma vez; vértices podem ser repetidos. A pilha permite percorrer novos trechos e encaixá-los na posição correta quando não há mais arestas disponíveis.

No triângulo AB, BC e CA, a saída pode ser A → B → C → A. Antes de aplicar o algoritmo a um grafo qualquer, verifique que todos os graus são pares e que todos os vértices de grau positivo estão conectados.

Questão 16 — Euleriano, hamiltoniano e grafo de linha

ENUNCIADO

Avalie cinco afirmações envolvendo grafos eulerianos, hamiltonianos e o grafo de linha L(G).

RESPOSTA DE PROVA

Definição: cada aresta de G se torna um vértice de L(G). Dois vértices de L(G) são adjacentes quando as arestas correspondentes compartilham um extremo em G.

1) “Se G é euleriano, então G é hamiltoniano.” Falso. Considere dois triângulos que compartilham somente um vértice x. Os graus são 4 em x e 2 nos outros vértices; o grafo é conexo e euleriano. Porém, x é uma articulação e não há ciclo que visite todos os vértices exatamente uma vez: seria necessário passar por x para transitar entre os dois lados e voltar.

2) “Se G é hamiltoniano, então G é euleriano.” Falso. K4 tem um ciclo hamiltoniano, mas cada vértice possui grau 3. Como há graus ímpares, não é euleriano.

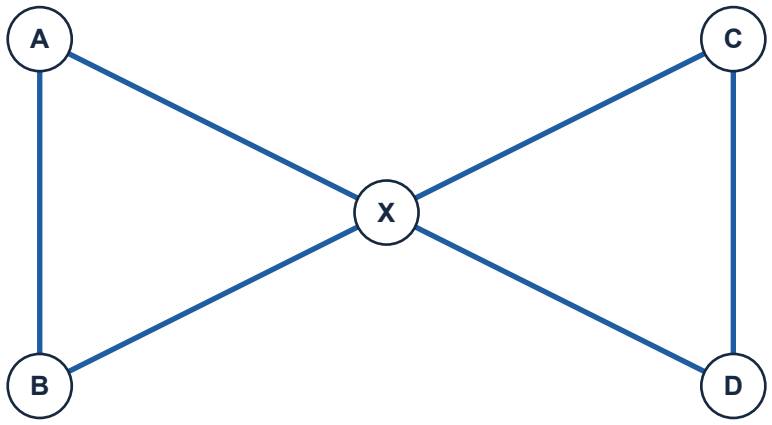
3) “Se G é euleriano, então L(G) é hamiltoniano.” Verdadeiro, considerando pelo menos três arestas. Escreva um ciclo euleriano de G como uma sequência e1, e2, ..., em, com retorno ao início. Cada aresta aparece exatamente uma vez e arestas consecutivas compartilham um extremo, inclusive a última e a primeira. Em L(G), essa sequência é um ciclo que visita todos os seus vértices exatamente uma vez.

4) “L(L(G)) = G.” Falso em geral, mesmo interpretando igualdade como isomorfismo. Tome G = P4, o caminho com quatro vértices. Então L(G) = P3 e L(L(G)) = P2. Os números de vértices são diferentes, logo os grafos não são isomorfos.

5) “Se G é hamiltoniano, então L(G) é euleriano.” Falso. Tome o ciclo A–B–C–D–A e acrescente a diagonal AC. G continua hamiltoniano. Os graus de A e C são 3, enquanto os de B e D são 2. O vértice de L(G) correspondente a AB possui grau 3 + 2 − 2 = 3, que é ímpar. Logo, L(G) não é euleriano.

Questão 16 — Euleriano, hamiltoniano e grafo de linha · continuação

Item 1: euleriano, mas não hamiltoniano



EXPLICAÇÃO

Euleriano trata de usar todas as arestas uma vez. Hamiltoniano trata de visitar todos os vértices uma vez. O grafo de linha conecta essas duas ideias porque transforma arestas em vértices, mas as implicações não podem ser invertidas automaticamente.

Para memorizar os cinco itens, na ordem: falso, falso, verdadeiro, falso, falso. Os contraexemplos e a construção do item 3 são a justificativa necessária para a prova.

Questão 17 — Quando o grafo de linha é euleriano ou hamiltoniano?

ENUNCIADO

Determine condições sobre G para que (a) L(G) seja euleriano e (b) L(G) seja hamiltoniano.

RESPOSTA DE PROVA

(a) Para uma aresta uv de um grafo simples, o grau do vértice correspondente em L(G) é:

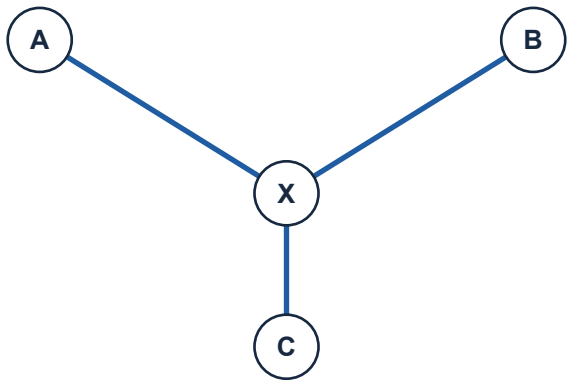
> **grau_L(uv) = grau_G(u) + grau_G(v) − 2.**

Logo, esse grau é par se, e somente se, u e v têm graus de mesma paridade. Portanto, L(G) é euleriano se seus vértices de grau positivo estiverem na mesma componente e, para toda aresta uv, os graus de u e v tiverem a mesma paridade. Se G for conexo e tiver pelo menos duas arestas, a conectividade de L(G) já está garantida. Nesse caso, a condição equivale a todos os vértices de G terem graus pares ou todos terem graus ímpares.

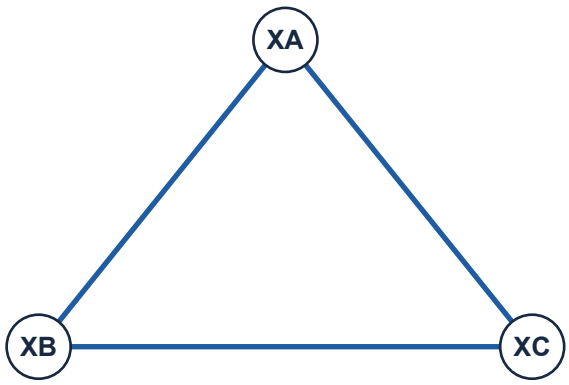
(b) Uma condição suficiente é G ser euleriano: seu ciclo euleriano fornece um ciclo hamiltoniano em L(G), como na questão 16. Porém, essa condição não é necessária. Na estrela K1,3, G não é euleriano, mas L(G) = K3 é hamiltoniano.

A caracterização completa, para G conexo com pelo menos três arestas, é: L(G) é hamiltoniano se, e somente se, G possui um subgrafo conexo H cujos graus são todos pares e cujo conjunto de vértices toca toda aresta de G. Aqui, H pode ser apenas um vértice. Trata-se do teorema de Harary e Nash-Williams, também expresso pela existência de uma trilha fechada dominante. [1]

G = K1,3



L(G) = K3



EXPLICAÇÃO

No item (a), a aresta uv é vizinha, no grafo de linha, das outras grau(u) − 1 arestas de u e das outras grau(v) − 1 arestas de v. Somando, obtemos a fórmula.

Questão 17 — Quando o grafo de linha é euleriano ou hamiltoniano? · continuação

Por isso, não é necessário que G tenha apenas graus pares. Na estrela K1,3, os graus são 3, 1, 1 e 1: todos ímpares. Seu grafo de linha é um triângulo, que é euleriano e hamiltoniano.

No item (b), a palavra “dominante” significa que cada aresta de G tem pelo menos um extremo em H. O subgrafo H oferece uma trilha fechada para organizar as arestas. No exemplo da estrela, H pode ser somente o centro: seu grau em H é zero, e ele toca todas as arestas da estrela.

O resultado avançado não deve ser confundido com “G precisa ser euleriano”. Para uma resposta baseada apenas no conteúdo elementar, apresente a condição suficiente, a prova da questão 16 e o contraexemplo que mostra que ela não é necessária.

Questão 18 — Aplicações de grafos eulerianos

ENUNCIADO

Cite aplicações modeladas por grafos não direcionados em que percursos eulerianos sejam necessários.

RESPOSTA DE PROVA

Podemos modelar interseções ou pontos de conexão como vértices e trechos que devem ser atendidos como arestas. Exemplos são: inspeção de todas as ruas de uma rede; coleta de resíduos em todos os trechos; inspeção de tubulações ou cabos; e desenho de uma figura sem levantar o lápis nem repetir segmentos.

Quando a tarefa exige percorrer cada aresta exatamente uma vez e retornar à origem, precisamos de um ciclo euleriano. Ele existe se, e somente se, todos os vértices de grau positivo estiverem conectados e todos os graus forem pares. Se for permitido terminar em outro vértice, um percurso euleriano aberto existe quando há exatamente dois vértices ímpares, além da mesma condição de conectividade.

Se a rede não satisfizer essas condições e for permitido repetir trechos, o modelo passa a ser o problema do carteiro chinês: buscamos repetir arestas com o menor custo adicional para cobrir toda a rede.

EXPLICAÇÃO

Esses problemas pedem visitar arestas, não apenas vértices. Visitar todas as esquinas não garante percorrer todas as ruas. Essa é a diferença em relação aos problemas hamiltonianos.

Por exemplo, um veículo que precisa inspecionar cada rua exatamente uma vez e voltar à garagem necessita de um ciclo euleriano. Se houver ruas que inevitavelmente precisam ser repetidas, ainda podemos planejar uma rota de cobertura, mas ela não será um ciclo euleriano do grafo original.

Referências e observações finais

Fonte dos exercícios: Grafos/AtividadesSala/AtividadesSala.txt, seção “Questões de sala até a P2”. As respostas foram elaboradas para este caderno; o arquivo original foi preservado.

[1] Harary, F.; Nash-Williams, C. St. J. A. On Eulerian and Hamiltonian Graphs and Line Graphs. Canadian Mathematical Bulletin. Referência da caracterização citada na questão 17(b):

Consultar o artigo original (Cambridge University Press).

Para revisão: BFS minimiza quantidade de arestas; Dijkstra minimiza custo com pesos não negativos; Bellman–Ford admite pesos negativos; Prim e Kruskal minimizam o custo total de uma árvore geradora; Hierholzer percorre cada aresta exatamente uma vez.